

<2026.03.10>

ASD 특허미팅

JaeSeong Hong, Yu Rang Park, Keun-Ah Cheon
Department of Biomedical Systems Informatics
Yonsei University College of Medicine

Severance

전체 Flow

1. 데이터 세팅 : 예측하려는 데이터를 형식에 맞게 추가
2. 모델 예측 수행 : 이미지 전처리 및 실제 모델 추론
3. 결과 확인 : 예측 결과 확인

1. 데이터 세팅

예측하려는 데이터를 형식에 맞게 추가

- (1) ASD는 환자 단위로 발생하기 때문에, 좌안 / 우안의 예측 결과의 평균을 활용
- (2) 데이터는 images 경로 아래 환자 / 안저 방향 단위로 추가해야 합니다.
- (3) 단안 촬영에만 성공했을 경우, 한쪽만 추가하여도 무관합니다.

예시 tree 구조

Patient

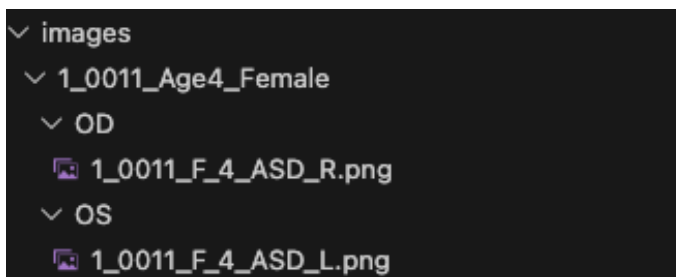
├── OD

│ └── fundus_image.png

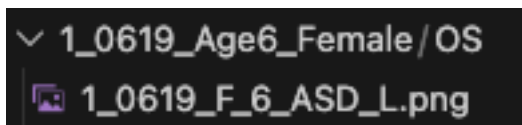
└── OS

└── fundus_image.png

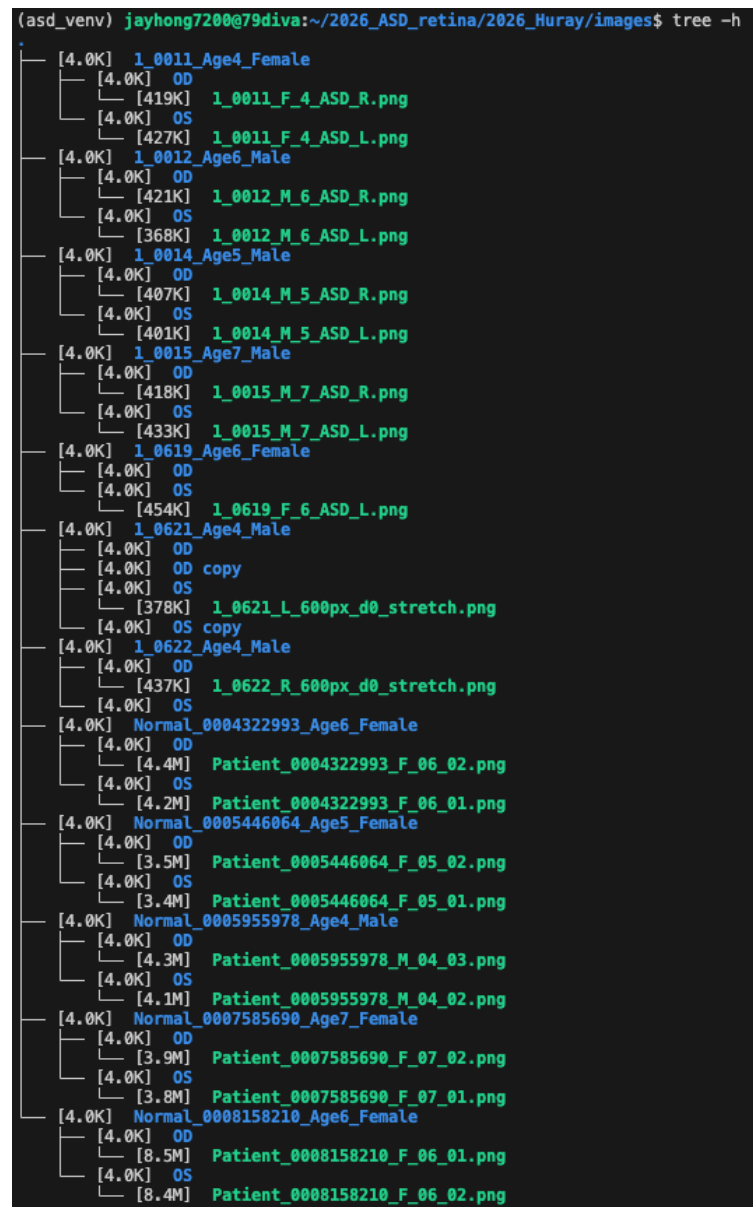
실제 예시 (1) - 양안 성공



실제 예시 (2) - 단안 성공



실제 예시 (3) : 전체 tree구조



2. 모델 예측 수행

모델 예측 방법 :

1단계 데이터 세팅 완료 후,

Bash script에서 ``python main.py`` 입력

Main.py 내부에서는 4단계에 걸쳐 예측 수행

1단계 : 예측을 위한 모델 불러오기

2단계 : 예측을 수행할 데이터 확인

3단계 : 데이터 전처리 및 예측 수행

4단계 : 예측 결과 저장

```
def main() :  
    # 1. 예측을 위한 모델 로드  
    model_paths = [f'models/Normal_ASD_fold_{i}.pth' for i in range(9)]  
    models = load_models(model_paths)  
  
    # 2. 예측을 수행할 폴더 확인  
    image_root_dir = Path('images')  
    image_dirs = [d for d in image_root_dir.glob('*') if d.is_dir()]  
    image_dirs = sorted(image_dirs, key=lambda x: x.name)  
  
    # 3. 각 폴더에 대해서 예측 수행  
    save_log = []  
    for image_dir in tqdm(image_dirs) :  
        result = process_single_dir(models, image_dir)  
        if result is None :  
            continue  
        asd_prob, final_prob = result  
        save_log.append({  
            'Patient_ID': image_dir.name,  
            'Left_Eye_ASD_Prob': asd_prob[0][1],  
            'Right_Eye_ASD_Prob': asd_prob[1][1],  
            'Final_ASD_Prob': final_prob  
        })  
  
    # 4. 결과 저장  
    save_log_path = Path('asd_prob_log.csv')  
    import pandas as pd  
    df = pd.DataFrame(save_log)  
    df.to_csv(save_log_path, index=False)  
    print(f"ASD probabilities saved to {save_log_path}")
```

2. 모델 예측 수행

1단계 : 예측을 위한 모델 불러오기

ASD 논문에서는, ensemble 이라는 방법을 사용하여 예측함.

- 서로 다른 데이터로 학습한 10개의 모델로 예측
- 예측 결과를 평균내어 안저 이미지에 대하여 평가
- 10개의 모델은

models/ 경로에 Normal_ASD_fold_*.pth 포맷으로 저장됨

- Main.py에서는 load_models로 모델을 불러와서 활용함.

```
def main() :  
    # 1. 예측을 위한 모델 로드  
    model_paths = [f'models/Normal_ASD_fold_{i}.pth' for i in range(9)]  
    models = load_models(model_paths)
```

```
def load_model(model_path) :  
    model = resnext_scratch(2)  
    state_dict = torch.load(model_path, map_location='cpu')  
    model.load_state_dict(state_dict)  
    return model  
  
def load_models(model_paths) :  
    models = []  
    for model_path in model_paths :  
        model = load_model(model_path).to('cuda').eval()  
        models.append(model)  
    return models
```

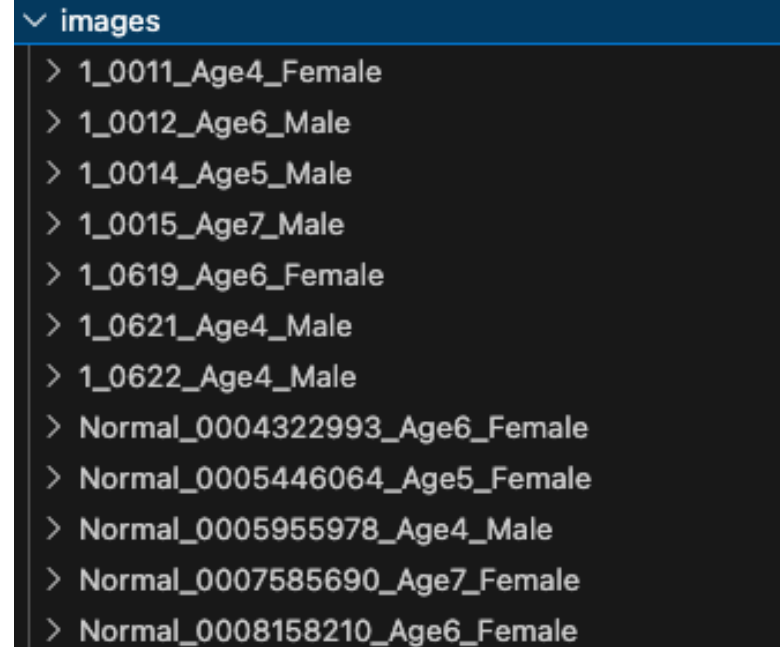
```
(asd_venv) jayhong720@79d1va:~/2026_ASD_retina/2026_Huray/models$ tree -h  
.  
├── [ 88M] Normal_ASD_fold_0.pth  
├── [ 88M] Normal_ASD_fold_1.pth  
├── [ 88M] Normal_ASD_fold_2.pth  
├── [ 88M] Normal_ASD_fold_3.pth  
├── [ 88M] Normal_ASD_fold_4.pth  
├── [ 88M] Normal_ASD_fold_5.pth  
├── [ 88M] Normal_ASD_fold_6.pth  
├── [ 88M] Normal_ASD_fold_7.pth  
├── [ 88M] Normal_ASD_fold_8.pth  
└── [ 88M] Normal_ASD_fold_9.pth  
  
0 directories, 10 files
```

2. 모델 예측 수행

2단계 : 예측을 수행할 데이터 확인

Images 폴더 내부에 환자 번호들을 로딩

```
# 2. 예측을 수행할 폴더 확인
image_root_dir = Path('images')
image_dirs = [d for d in image_root_dir.glob('*') if d.is_dir()]
image_dirs = sorted(image_dirs, key=lambda x: x.name)
```



- images
 - > 1_0011_Age4_Female
 - > 1_0012_Age6_Male
 - > 1_0014_Age5_Male
 - > 1_0015_Age7_Male
 - > 1_0619_Age6_Female
 - > 1_0621_Age4_Male
 - > 1_0622_Age4_Male
 - > Normal_0004322993_Age6_Female
 - > Normal_0005446064_Age5_Female
 - > Normal_0005955978_Age4_Male
 - > Normal_0007585690_Age7_Female
 - > Normal_0008158210_Age6_Female

2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

하나의 directory는 process_single_dir 함수에서 시행하며

해당 파일에서 좌안/우안의 정보를 받아고,

preprocess(image_path) 를 통해 데이터 전처리 수행

```
# 3. 각 폴더에 대해서 예측 수행
save_log = []
for image_dir in tqdm(image_dirs) :
    result = process_single_dir(models, image_dir)
    if result is None :
        continue
    asd_prob, final_prob = result
    save_log.append({
        'Patient_ID': image_dir.name,
        'Left_Eye_ASD_Prob': asd_prob[0][1],
        'Right_Eye_ASD_Prob': asd_prob[1][1],
        'Final_ASD_Prob': final_prob
    })
```

```
def process_single_dir(models, image_dir) :
    left_eye = list(image_dir.joinpath('0S').glob('*.png'))
    right_eye = list(image_dir.joinpath('0D').glob('*.png'))
    if len(left_eye) == 0 and len(right_eye) == 0 :
        print(f"No images found in {image_dir}")
        return None

    left_eye = ['Left', left_eye[0]] if len(left_eye) > 0 else ['Left', None]
    right_eye = ['Right', right_eye[0]] if len(right_eye) > 0 else ['Right', None]

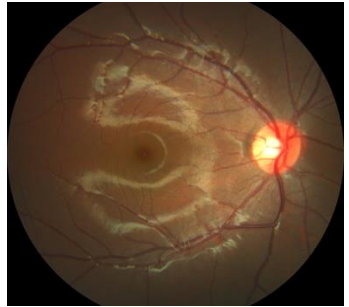
    asd_prob = []
    for eye, image_path in [left_eye, right_eye] :
        if image_path is None :
            asd_prob.append([eye, None])
            continue
        image_tensor = preprocess(image_path)
        image_tensor = image_tensor.unsqueeze(0).to('cuda')
        with torch.no_grad() :
            outputs = []
            for model in models :
                output = model(image_tensor)
                outputs.append(output)
            outputs = torch.stack(outputs, dim=0)
            avg_output = torch.mean(outputs, dim=0)
            avg_output = torch.softmax(avg_output, dim=1)
            asd_prob.append([eye, avg_output[0][1].item()])

    # final -> avg
    final_prob = np.mean([prob[1] for prob in asd_prob if prob[1] is not None])
    return asd_prob, final_prob
```


2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

데이터 전처리는 ASD 논문의 구조에 따라 다음과 같이 수행



Raw-image → Image Stretch → Fix RGB → Circle-mask → Crop-box → Square mask → Crop-height



2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

데이터 전처리는 ASD 논문의 구조에 따라 다음과 같이 수행

(1) Raw image :

원본 이미지는 cv2 패키지를 통해 로딩

(2) Stretch image :

이미지는 0~255사이의 값을 가지도록 stretch

* 이 외의 전처리 도구는 src/image_prep.py에 있음

```
image_raw = _load_image(image_path)
image_stretch = _stretch_image(image_raw)
prep_image = _preprocess(image_stretch)

image_save_path = prep_image_save_dir / image_path.parent.parent.name / image_path.parent.name / image_path.name
image_save_path.parent.mkdir(parents=True, exist_ok=True)
img_tensor = _image_to_tensor(prep_image)
return img_tensor

def _preprocess(image) :
    # step1 : get mask :
    image = maybe_fix_bgr_rgb(image) # 가끔씩 RGB가 잘못 들어오는 경우가 있어서 수정
    mask, bbox, _, _ = get_mask(image) # mask : 0,1로 이루어진 binary mask, bbox : (s_h, s_w, height, width)
    prep_image = mask_image(image, mask) # mask가 0인 부분은 검정색으로 바꿔줌
    prep_image, _ = remove_black_area(prep_image, bbox=bbox) # bbox를 이용해서 눈 영역만 남기고 나머지 부분은 제거
    prep_image, _ = supplemental_black_area(prep_image) # 눈 영역이 정상각형이 되도록 보조적으로 검정색 영역을 추가

    # 하단이나 상단에 환자 정보가 적혀있는 경우가 있어서, 강제로 crop 후 학습에 활용하였음.
    height = prep_image.shape[0] # 눈 영역의 높이
    prep_image = prep_image[int(height*0.05):int(height*0.95)]
    return prep_image
```

```
def _load_image(image_path) :
    image = cv2.imread(str(image_path))
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    return image
```

```
def _stretch_image(image) :
    # min max scaling to [0,255]
    min_val = np.min(image)
    max_val = np.max(image)
    stretched_image = (image - min_val) / (max_val - min_val) * 255
    return stretched_image.astype(np.uint8)
```

2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

데이터 전처리는 ASD 논문의 구조에 따라 다음과 같이 수행

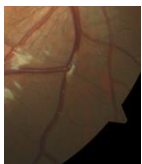
(3) Fix RGB

maybe_fix_bgr_rgb(image)를 통해 처리

(4) Circle mask

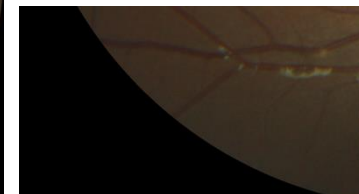
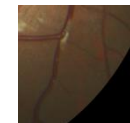
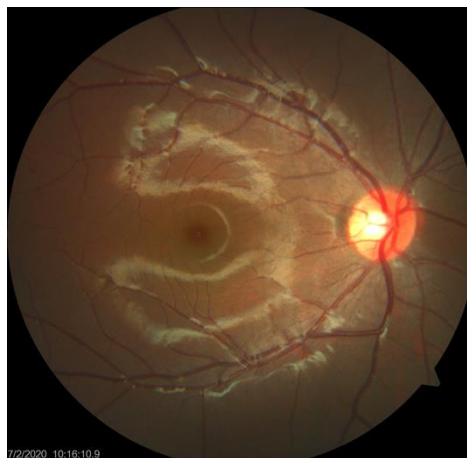
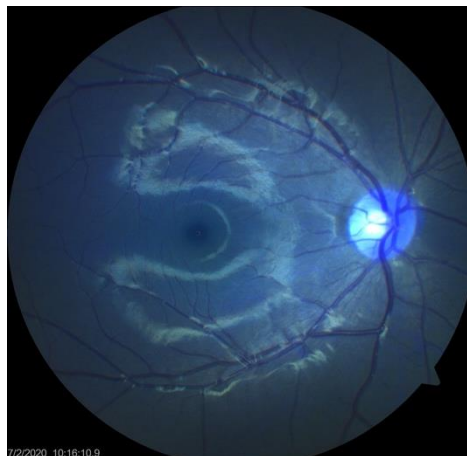
get_mask와 mask_image 함수를 통해 처리

*안저 영역 이외의 non-informative한 영역은 삭제



```
def _preprocess(image) :
    # step1 : get mask :
    image = maybe_fix_bgr_rgb(image) # 가끔씩 RGB가 잘못 들어오는 경우가 있어서 수정
    mask, bbox, _, _ = get_mask(image) # mask : 0,1로 이루어진 binary mask, bbox : (s_h, s_w, height, width)
    prep_image = mask_image(image, mask) # mask가 0인 부분은 검정색으로 바꿔줌
    prep_image, _ = remove_back_area(prepare_image, bbox=bbox) # bbox를 이용해서 눈 영역만 남기고 나머지 부분은 제거
    prep_image, _ = supplemental_black_area(prepare_image) # 눈 영역이 정사각형이 되도록 보조적으로 검정색 영역을 추가

    # 하단이나 상단에 환자 정보가 적혀있는 경우가 있어서, 강제로 crop 후 학습에 활용하였음.
    height = prep_image.shape[0] # 눈 영역의 높이
    prep_image = prep_image[int(height*0.05):int(height*0.95)]
    return prep_image
```



2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

데이터 전처리는 ASD 논문의 구조에 따라 다음과 같이 수행

```
def _preprocess(image) :
    # step1 : get mask :
    image = maybe_fix_bgr_rgb(image) # 가끔씩 RGB가 잘못 들어오는 경우가 있어서 수정
    mask, bbox, _, _ = get_mask(image) # mask : 0,1로 이루어진 binary mask, bbox : (s_h, s_w, height, width)
    prep_image = mask_image(image, mask) # mask가 0인 부분은 검정색으로 바꿔줌
    prep_image, _ = remove_black_area(prepare_image, bbox=bbox) # bbox를 이용해서 눈 영역만 남기고 나머지 부분은 제거
    prep_image, _ = supplemental_black_area(prepare_image) # 눈 영역이 정사각형이 되도록 보조적으로 검정색 영역을 추가

    # 하단이나 상단에 환자 정보가 적혀있는 경우가 있어서, 강제로 crop 후 학습에 활용하였음.
    height = prep_image.shape[0] # 눈 영역의 높이
    prep_image = prep_image[int(height*0.05):int(height*0.95)]
    return prep_image
```

(5) Crop box

테두리 영역의 검정 부분을 삭제

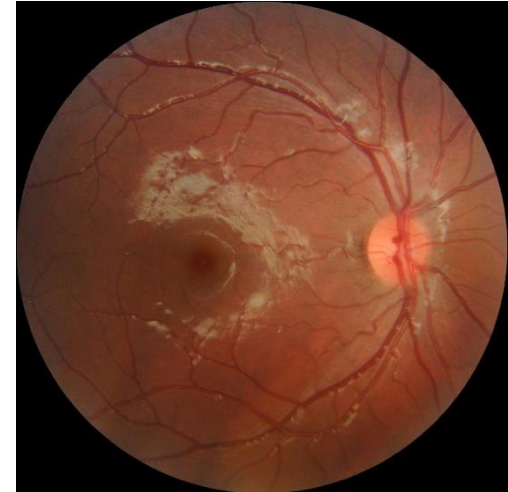
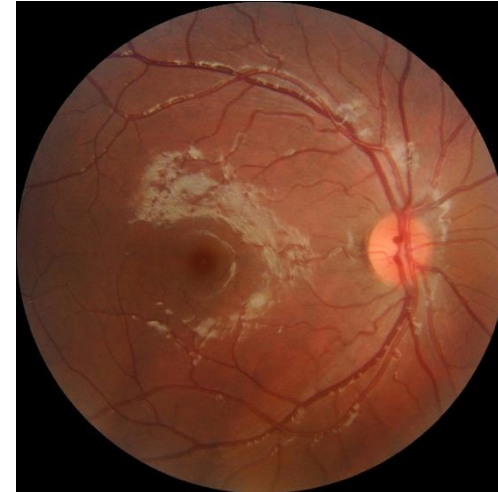
non-informative 영역을 masking하면서 발생한

검정 테두리 부분을 제거

(6) Square mask

직사각형 형태로 crop된 이미지를

정사각형 형태로 다시 mask 추가



2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

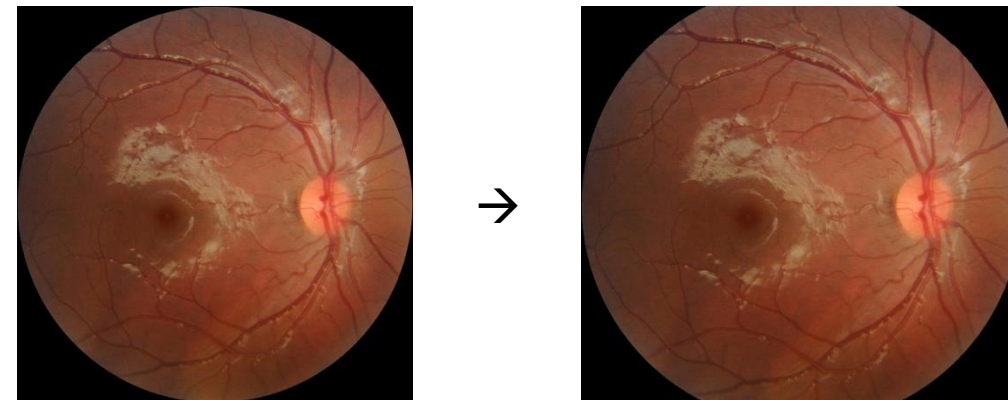
데이터 전처리는 ASD 논문의 구조에 따라 다음과 같이 수행

(7) Crop-height

이미지의 top-bottom을 5%씩 잘라내어 활용

```
def _preprocess(image) :
    # step1 : get mask :
    image = maybe_fix_bgr_rgb(image) # 가끔씩 RGB가 잘못 들어오는 경우가 있어서 수정
    mask, bbox, _, _ = get_mask(image) # mask : 0,1로 이루어진 binary mask, bbox : (s_h, s_w, height, width)
    prep_image = mask_image(image, mask) # mask가 0인 부분은 검정색으로 바꿔줌
    prep_image, _ = remove_black_area(prep_image, bbox=bbox) # bbox를 이용해서 눈 영역만 남기고 나머지 부분은 제거
    prep_image, _ = supplemental_black_area(prep_image) # 눈 영역이 정사각형이 되도록 보조적으로 검정색 영역을 추가

    # 하단이나 상단에 환자 정보가 적혀있는 경우가 있어서, 강제로 crop 후 학습에 활용하였음.
    height = prep_image.shape[0] # 눈 영역의 높이
    prep_image = prep_image[int(height*0.05):int(height*0.95)]
    return prep_image
```



2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

전처리된 이미지는 “Preprocessed_Images” 경로에 저장됨 (전처리 결과 확인용)



2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

최종 이미지는 딥러닝 모델의 포맷에 맞게 tensor로 변경

원본 이미지는 (224,224) 사이즈로 변경되며,

Tensor 형식으로 변환됨

```
def _image_to_tensor(image) :  
    transform = transforms.Compose([  
        transforms.ToPILImage(),  
        transforms.Resize((224,224)),  
        transforms.ToTensor(),  
    ])  
    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)  
    return transform(image)
```

2. 모델 예측 수행

3단계 : 데이터 전처리 및 예측 수행

모델 예측 수행

- (1) 좌안/우안 이미지에 대하여, 각 이미지 마다
- (2) 10개의 model을 돌며 예측 수행, 결과를 outputs에 저장
- (3) 10개 모델에 대한 평균 수행, avg_output에 저장
- (4) 좌안/우안의 정보 각각이 asd_prob에 저장되며,
최종 예측은 좌/우안 예측 결과의 평균 활용

```
with torch.no_grad() :  
    outputs = []  
    for model in models :  
        output = model(image_tensor)  
        outputs.append(output)  
    outputs = torch.stack(outputs, dim=0)  
    avg_output = torch.mean(outputs, dim=0)  
    avg_output = torch.softmax(avg_output, dim=1)  
    asd_prob.append([eye, avg_output[0][1].item()])  
# final -> avg  
final_prob = np.mean([prob[1] for prob in asd_prob if prob[1] is not None])  
return asd_prob, final_prob
```


2. 모델 예측 수행

4단계 : 예측 결과 저장

한 사람에 대한 최종 예측 결과는 아래와 같이 구성됨

환자 번호 / 좌안 예측 결과 / 우안 예측 결과 / 최종 예측 결과

모든 환자에 대한 결과는 asd_prob_log.csv 에 저장됨

```
save_log.append({
    'Patient_ID': image_dir.name,
    'Left_Eye_AS_Prob': asd_prob[0][1],
    'Right_Eye_AS_Prob': asd_prob[1][1],
    'Final_AS_Prob': final_prob
})
```

```
# 4. 결과 저장
save_log_path = Path('asd_prob_log.csv')
import pandas as pd
df = pd.DataFrame(save_log)
df.to_csv(save_log_path, index=False)
print(f"ASD probabilities saved to {save_log_path}")
```

3. 결과 확인

4단계 : 예측 결과 저장

최종 결과는 csv 파일을 다운받아서 확인

단안 촬영 결과만 있을 경우에는, NaN 값으로 표기됨

	Patient_ID	Left_Eye_ASD_Prob	Right_Eye_ASD_Prob	Final_ASD_Prob
0	1_0011_Age4_Female	0.960907	0.923624	0.942266
1	1_0012_Age6_Male	1.000000	0.987717	0.993858
2	1_0014_Age5_Male	0.744701	0.990235	0.867468
3	1_0015_Age7_Male	0.984447	0.997475	0.990961
4	1_0619_Age6_Female	0.797925	NaN	0.797925
5	1_0621_Age4_Male	0.999609	NaN	0.999609
6	1_0622_Age4_Male	NaN	0.999017	0.999017
7	Normal_0004322993_Age6_Female	0.000070	0.000001	0.000035
8	Normal_0005446064_Age5_Female	0.000028	0.000033	0.000030
9	Normal_0005955978_Age4_Male	0.000002	0.000045	0.000023
10	Normal_0007585690_Age7_Female	0.002101	0.000592	0.001346
11	Normal_0008158210_Age6_Female	0.100109	0.104198	0.102154